MODULE 23 · EXAM STRATEGY - ALL BLOCKS

Confusion pairs

The look-alikes the exam puts side by side, and a one-line rule for telling each pair apart.

BLOCKS all four FOCUS look-alikes MODULE 23 of 28

admin@sthithapragna.com

When two options differ by one small thing, the question is **testing that difference**. Find it, apply its rule, and the answer follows.

Operators that look alike

PAIR	THE RULE	EXAMPLE
`/ versus //`	/ always gives a float; // floors	7 / 2 is 3.5; 7 // 2 is 3
`// versus int()`	// rounds down; int() cuts towards zero	-7 // 2 is -4; int(-3.5) is -3
`= versus ==`	= assigns; == compares	if x = 5: is a SyntaxError
`and versus &`	and returns an operand; & works on bits	1 and 2 is 2; 1 & 2 is 0
`is versus ==`	is: same object; ==: equal values	[1] == [1] True; [1] is [1] False
`"2" + "4" versus 2 + 4`	+ joins strings, adds numbers	"24" versus 6

Control flow that looks alike

PAIR	THE RULE	EXAMPLE, OVER "AB3CD" STOPPING AT 3
`break`	leaves the loop at once	ab
`continue`	skips to the next pass	abcd
`pass`	does nothing at all	ab3cd
`elif chain versus separate ifs`	one branch at most; each if decides alone	count the chains first
`loop else versus if else`	loop else: no break happened	runs even after zero passes
`range(5) versus range(5, 0)`	one argument is stop; no step counts up	0 to 4, versus empty

List operations that look alike

PAIR	THE RULE	EXAMPLE, FROM [1, 2]
`append versus extend`	append adds one element; extend adds each	[1, 2, [3]] versus [1, 2, 3]
`sort() versus sorted()`	sort changes the list, returns None; sorted returns a new list	n.sort() is None
`pop() versus remove() versus del`	pop by position and returns; remove by value; del by position	pop() returns the element
`b = a versus b = a[:]`	alias shares one list; slice copies it	a change through b shows in a
`[[0]*2]*2 versus a comprehension`	* repeats one inner list; a comprehension builds new ones	[[1, 0], [1, 0]] after one change
`x += [3] versus x = x + [3]`	+= changes the shared list; = + builds a new one	matters only with an alias

Tuples, strings and dictionaries

PAIR	THE RULE	EXAMPLE
`(1) versus (1,)`	the comma makes a tuple	int versus tuple
`find() versus index()`	find gives -1; index raises ValueError	"banana".find("x") is -1
`d[k] versus d.get(k)`	[] raises KeyError; get gives None or a default	{"a": 1}.get("b") is None
`k in d versus v in d.values()`	in on a dict checks keys only	1 in {"a": 1} is False
`s.upper() versus s = s.upper()`	a string method's result must be kept	s itself never changes
`len() versus count()`	len counts everything; count one value	[1, 1, 2]: 3 versus 2

Functions and exceptions that look alike

PAIR	THE RULE	EXAMPLE
`return versus print`	return gives a value back; print only shows it	print(print("x")) shows x, then None
`local versus global`	assigned in the function: local, unless global	total += n needs global total
`f(a=1, 2) versus f(1, a=2)`	wrong order: SyntaxError; filled twice: TypeError	one never runs; one fails at the call
`TypeError versus ValueError`	wrong type, versus right type with a bad value	int(None) versus int("x")
`IndexError versus KeyError`	bad position, versus missing dictionary key	[1][5] versus {}["k"]
`SyntaxError versus runtime error`	nothing runs, versus stops at the failing line	partial output only for runtime

admin@sthithapragna.com

break, continue and pass on the same loop

```
1  for word in ("break", "continue", "pass"):
2      out = ""
3      for ch in "ab3cd":
4          if ch == "3":
5              if word == "break": break
6              elif word == "continue": continue
7              else: pass
8          out += ch
9      print(word, out)     # break ab, continue abcd, pass ab3cd
```

5 break: the rest of the string is never reached.

6 continue: only the 3 is skipped.

7 pass: nothing happens, so the 3 is added like any other character.

What to carry into the exam

- Find the **one difference** between the options, then apply that pair's rule.
- // floors; int() cuts to zero. append adds one; extend adds each.
- sort() returns None; sorted() returns a list. find gives -1; index raises.
- The comma makes a tuple. in on a dict checks keys. pass does nothing.
- Wrong argument order: SyntaxError. Wrong count or filled twice: TypeError.

What is the output of this program?

```
1  a = [1, 2]
2  b = a
3  c = a[:]
4  a += [3]
5  c.append(4)
6  print(len(a), len(b), len(c))
```

A 3 2 3

B 3 3 3

C 3 3 2

D 2 2 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [1, 2]
2  b = a
3  c = a[:]
4  a += [3]
5  c.append(4)
6  print(len(a), len(b), len(c))
```

a and b share [1, 2, 3]. c is a separate [1, 2, 4].

A **3 2 3**

b is an alias, so it sees a's += as well.

B **3 3 3**

+= grows the shared list to 3; the copy grows separately to 3.

C **3 3 2**

c's own append makes it 3 long too.

D **2 2 3**

+= changes the list a and b share.

THE SIGNAL

Alias versus copy, and += changing the shared list: two pairs in one question.

What is the output of this program?

```
out = ""
for ch in "a1b2":
    if ch.isdigit():
        continue
    out += ch
for ch in "a1b2":
    if ch.isdigit():
        pass
    out += ch
print(out)
```

A ab

B abab

C aba1b2

D a

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  out = ""
2  for ch in "a1b2":
3      if ch.isdigit():
4          continue
5      out += ch
6  for ch in "a1b2":
7      if ch.isdigit():
8          pass
9      out += ch
10 print(out)
```

Loop 1 skips digits: "ab". Loop 2 keeps everything: "a1b2".

A `ab`

Forgets the second loop adds to the same out.

B `abab`

Treats pass like continue. pass skips nothing.

C `aba1b2`

continue skips the digits; pass does nothing, so they are kept.

D `a`

Treats continue like break.

THE SIGNAL

continue skips the rest of the pass. pass does nothing at all.

What is the output of this program?

```
1  x = -7
2  print(x / 2, x // 2, int(x / 2), round(x / 2))
```

A -3.5 -4 -3 -4

B -3.5 -3 -3 -4

C -3.5 -4 -4 -4

D -3.5 -4 -3 -3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = -7
2  print(x / 2, x // 2, int(x / 2), round(x / 2))
```

-3.5 exactly. Floor: -4. Truncate: -3. Round half to even: -4.

A `-3.5 -4 -3 -4`

/ exact; // floors down; int cuts to zero; round goes to the even -4.

B `-3.5 -3 -3 -4`

// rounds down, towards minus infinity. -4.

C `-3.5 -4 -4 -4`

int() cuts towards zero: -3.

D `-3.5 -4 -3 -3`

round(-3.5) goes to the even neighbour, -4.

THE SIGNAL

Four ways to turn -3.5 into a whole number, and three different answers.

What is the output of this program?

```
1  words = ["b", "a"]
2  x = words.sort()
3  y = sorted("cab")
4  print(x, y, "cab".find("z"))
```

A `['a', 'b'] ['a', 'b', 'c'] -1`

B `None abc -1`

C `None ['a', 'b', 'c'] -1`

D The code raises a ValueError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  words = ["b", "a"]
2  x = words.sort()
3  y = sorted("cab")
4  print(x, y, "cab".find("z"))
```

sort() returns None. sorted() always returns a list. find() returns -1.

A `['a', 'b'] ['a', 'b', 'c'] -1`
x holds sort's return value, which is None.

B `None abc -1`
sorted() returns a list, even from a string.

C `None ['a', 'b', 'c'] -1`
Three pairs: sort vs sorted, str vs list, find vs index.

D The code raises a ValueError
find returns -1. Only index would raise.

THE SIGNAL

sort returns None; sorted returns a list; find returns -1.

What is the output of this program?

```
1  a = (1)
2  b = (1,)
3  c = [1]
4  print(type(a) == int, len(b), c == [1], c is [1])
```

A `True 1 True True`

B `False 1 True False`

C The code raises a TypeError

D `True 1 True False`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = (1)
2  b = (1,)
3  c = [1]
4  print(type(a) == int, len(b), c == [1], c is [1])
```

(1) is the int 1. (1,) is a one-element tuple. [1] makes a new list each time.

A `True 1 True True`
Each [1] builds a new list, so is gives False.

B `False 1 True False`
(1) has no comma, so a is an int.

C The code raises a TypeError
a is an int, but nothing takes its len. b's len is fine.

D `True 1 True False`
No comma: int. Comma: tuple of length 1. Equal lists; different objects.

THE SIGNAL

The comma makes a tuple. == compares values; is compares identity.

> Each line is run as a whole program on its own.

Which two programs print the word None? (Choose two.)

A `print(print("x"))`

B `print({"a": 1}.get("b"))`

C `print([3, 1].sort() == None)`

D `print(len([None]))`

◆ PAUSE AND DECIDE

Which two programs print the word None? (Choose two.)

A `print(print("x"))`

Prints x, then the inner print's return value, None.

B `print({"a": 1}.get("b"))`

A missing key with no default: get returns None.

C `print([3, 1].sort() == None)`

sort does return None, but the comparison prints True.

D `print(len([None]))`

The list has one element, so this prints 1.

THE SIGNAL

Something that returns None only prints None if None itself is what gets printed.